

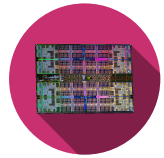
RISC-V Architecture

Lab Computer Architecture

Contents

1	Introduction	2
2	Single-cycle	3
2.1	Setup Ripes	3
2.2	Block mapping	3
2.3	Signal size and behavior	4
2.4	Architectural behavior	5
3	Limitations of the SCR architecture	7
4	Multi-cycle / Pipeline	8
4.1	Multi-cycle	8
4.2	Pipeline	9
4.3	Flush and hazards	9





1 | Introduction

This laboratory studies a simple [Reduced Instruction Set Computer \(RISC-V\)](#) core through three architectural organizations: single-cycle, multi-cycle, and pipelined. The goal is to understand how computation, memory and control blocks interact, and how each architectural choice affects execution time and throughput.

By the end of the lab, you should be able to:

- identify the main blocks of a simple RISC-V processor and explain their role
- map control/data signals to their size and behavioral effect
- estimate the execution timing of a **lw** instruction and derive a maximum clock frequency
- explain why migrating to a multi-cycle architecture, and later to pipelining, makes sense

The activities are split in three parts:

- [Section 2](#): discover a simplified single-cycle architecture
- [Section 3](#): estimate **lw** timing and discuss architectural tradeoffs
- [Section 4](#): inspect a 5-stage pipelined architecture without forwarding and hazard detection.

Single-cycle

A **SCR** processor executes one instruction per cycle. Therefore, every block needed by that instruction must finish before the next clock edge.

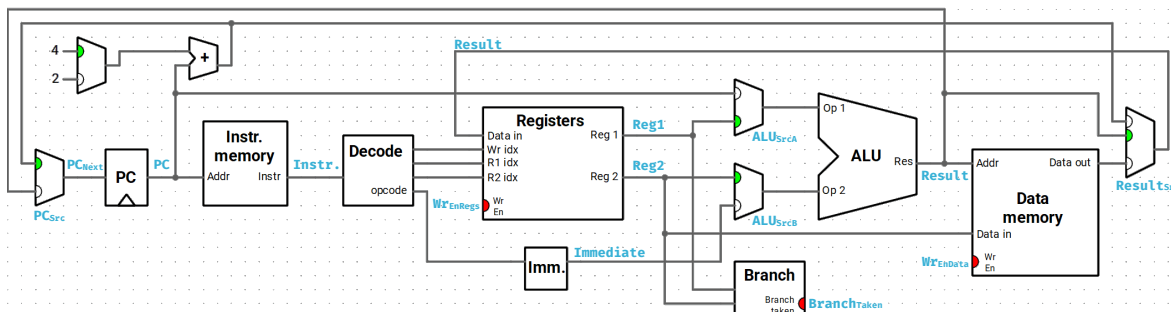


Figure 1 - Simplified single-cycle architecture (Ripes view)

2.1 Setup Ripes

Configure Ripes to use a “Single-cycle processor”. Under **Processor**, right-click -> **Ports-> Show all values** to show the signal values:

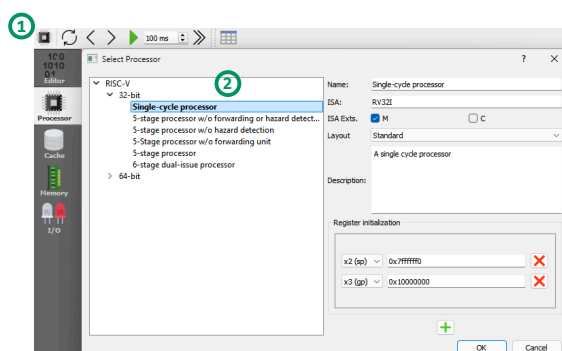


Figure 2 - Ripes setup for the single-cycle processor

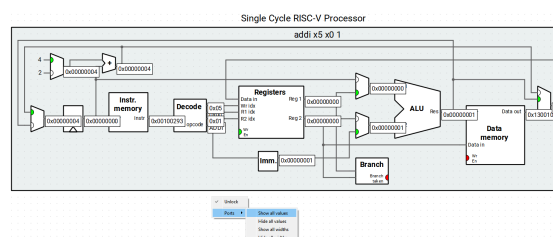
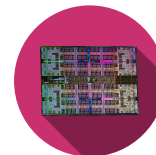


Figure 3 - Signal values display in Ripes

2.2 Block mapping

In [Figure 1](#), map each Ripes block to its architectural role:

- 



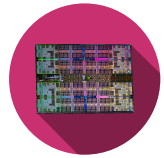
2.3 Signal size and behavior



Complete the following table from your Ripes observations

Signal	Source	Width in bits	Function
PC	Program Counter	32	Points to the current instruction in memory
Instr.	Instruction Memory		
Reg1	Registers		
Reg2	Registers		
Immediate	Imm.		
Result	ALU		

Table 1 - Guided signal analysis



2.4 Architectural behavior

Run a few instructions across different formats using the following code:

```

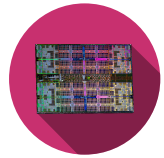
1  # Adapted from Ripes example:
2  # https://github.com/mortbopet/ripes/blob/master/examples/assembly/factorial.s
3  # Goal: demonstrate recursive function calls, JAL/RET, stack usage,
4  #       branches, and register/data flow.
5  # Calculates 5! = 5 * 4 * 3 * 2 * 1 = 120.
6  .data
7  argument: .word 5
8  .text
9  main:
10     lw  a0, argument      # a0 = argument (5)
11     jal ra, fact          # call fact(5)
12
13     # Display result
14     li  a7, 1
15     ecall
16
17     # Exit program
18     li  a7, 10
19     ecall
20
21     # Recursive factorial:
22     #   fact(n) = 1           if n < 1
23     #   fact(n) = n * fact(n-1) otherwise
24     # a0 = argument and return value
25     # Each call uses a 16-byte stack frame:
26     #   0(sp) = n
27     #   8(sp) = return address
28     fact:
29     li  t0, -16
30     add sp, sp, t0        # allocate 16-byte stack frame
31
32     sw  ra, 8(sp)         # save return address
33     sw  a0, 0(sp)         # save original n
34
35     addi t0, a0, -1       # t0 = n - 1
36     bge t0, zero, nfact   # if n >= 1, recurse
37
38     # Base case: return 1
39     addi a0, zero, 1
40     addi sp, sp, 16       # free stack frame
41     jr  x1               # return to caller
42
43     nfact:
44     addi a0, a0, -1       # calculate fact(n - 1)
45     jal ra, fact          # recursive call
46
47     addi t1, a0, 0        # t1 = fact(n - 1)
48
49     lw  a0, 0(sp)         # restore original n
50     lw  ra, 8(sp)         # restore return address
51     addi sp, sp, 16       # free stack frame
52
53     mul a0, a0, t1        # a0 = n * fact(n - 1)
54     ret                  # return result

```



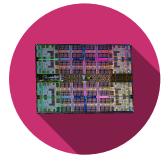
For instructions in [Table 2](#) at their first occurrence, record:

1. PC value before/after execution
2. ALU inputs/outputs
3. other active signals and muxes related to the instruction type



Instruction	PC before	PC after	ALU in1	ALU in2	ALU out	Active control signals
<code>add sp, sp, t0</code>	0x00000020	0x00000024	0x7FFFFFF0 = Reg1	0xFFFFFFFF0 = Reg1	0x7FFFFFFE0 = ADD	W_{EnRegs} $Result_{Src} = ALU$ $PC_{Next} = PC + 4$
<code>sw ra, 8(sp)</code>						
<code>addi t0, a0, -1</code>						
<code>bge t0, zero, nfact</code>						
<code>jr x1</code>						
<code>lw a0, 0(sp)</code>						
<code>mul a0, a0, t1</code>						

Table 2 - Analysis of executed instructions



3 Limitations of the SCR architecture

In this part, you will use the reference architecture as seen in class to estimate the critical timing of a **lw** instruction on a single-cycle processor.

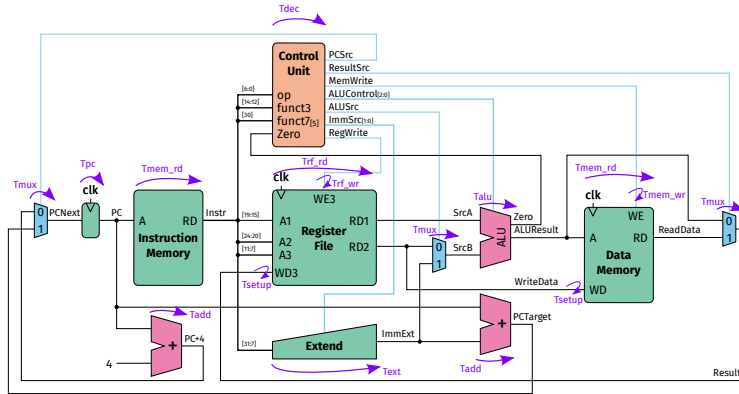


Figure 4 - Reference datapath for timing analysis

Block	Delay	Description
tMux	30 ps	Routing/muxing
tPC	40 ps	PC update
tAdd	100 ps	32-bit addition
tMemRd	200 ps	Memory read
tDec	25 ps	Decode
tRfRd	100 ps	Register read
tRfWr	60 ps	Register write
tSetup	50 ps	Register setup
tExt	35 ps	Immediate extension
tALU	120 ps	ALU operation
tMemWr	60 ps	Memory write

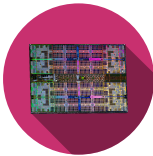
Table 3 - Per-block delay values

The purple arrows traversing blocks like T_{MemRd} represent the time it takes for the output to be valid after an input has changed.

The arrows on signals like T_{setup} represent the time required for an input value to be captured by a clock edge.



1. Trace the primary **lw** path from PC up to destination-register writeback (see Figure 6).
2. Compute total path delay and derive maximum frequency using $f_{\text{max}} = \frac{1}{T_{\text{cycle}}}$.
3. Discuss the single-cycle architecture pros and cons: simplicity, debugging, resources, frequency ...



4 | Multi-cycle / Pipeline

In this part, configure Ripes with a “5-stage processor w/o forwarding or hazard detection” - [Figure 2](#).

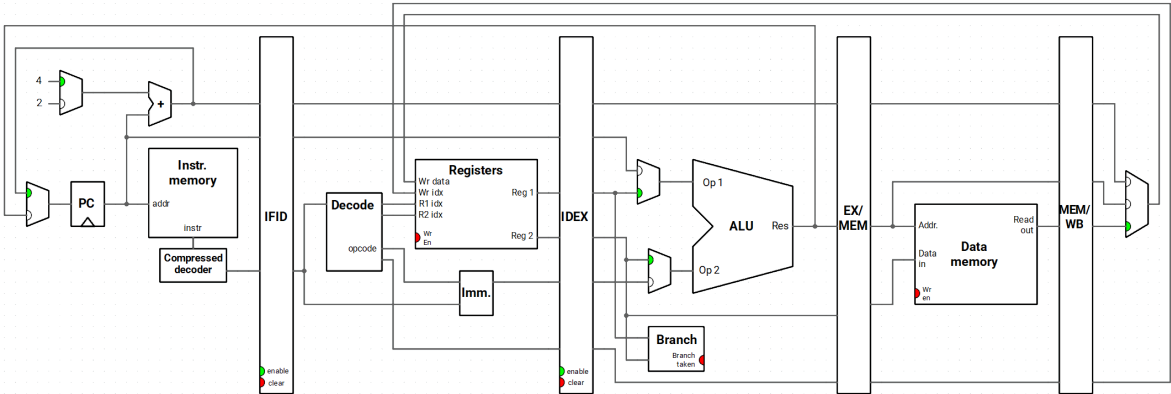


Figure 5 - 5-stage pipeline in Ripes

4.1 Multi-cycle

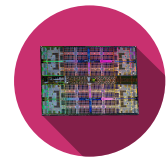
Formalize the operation of a **multi-cycle** architecture.



- 1. Using the [Table 4](#) as a guide, draw a timing diagram for a code containing, in order, **add**, **lw** and **sw**. For each clock cycle, indicate which block(s) execute which instruction(s).
- 2. For each instruction, indicate the latency (number of cycles per instruction).
- 3. Indicate the mean throughput of these three instructions.
- 4. What advantages does the multi-cycle architecture bring?

Etapes	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15
Fetch	add														
Decode															
Execute															
Memory															
Write-back															

Table 4 - Multi-cycle diagram



4.2 Pipeline

The pipeline architecture is inspired by the operation of multi-cycle, but improves throughput by executing multiple instructions in parallel.



1. Using the [Table 5](#) as a guide, draw a timing diagram for a code containing, in order, **add**, **lw** and **sw**. For each clock cycle, indicate which block(s) execute which instruction(s).
2. For each instruction, indicate the latency (number of cycles per instruction).
3. Indicate the mean throughput of these three instructions.
4. What advantages does the pipeline architecture bring? What disadvantages does it bring?

Etapes	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15
Fetch	add														
Decode															
Execute															
Memory															
Write-back															

Table 5 - Pipeline diagram

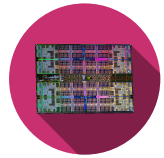
4.3 Flush and hazards

The pipeline introduces problems (**hazards**) when instructions depend on results from previous instructions. In this case, the pipeline must be “flushed” to avoid calculation errors.

This is especially the case for jump instructions, where previously loaded instructions must be discarded if the jump is taken.

Run the following code in Ripes while observing the state of the pipeline at each step:

```
# Ripes pipeline demonstration: branch / control-hazard penalty
#
# Purpose:
#   Show what happens to a pipeline when a conditional branch changes the
#   program counter.
#
# The loop executes three times:
#
#   s0 = 3 -> decrement to 2 -> branch TAKEN
#   s0 = 2 -> decrement to 1 -> branch TAKEN
#   s0 = 1 -> decrement to 0 -> branch NOT TAKEN
#
# In a pipelined processor, instructions following a branch may already have
# been fetched before the branch outcome is known. When the branch is taken,
# those incorrectly fetched instructions may have to be flushed.
#
# The NOPs after the loop are intentional. They make instructions following
# the branch visible in Ripes and help illustrate where execution continues.
```



```
#
# NOP is an alias for:
#   addi x0, x0, 0
#
# There is no meaningful program output; the interesting part is the
# pipeline timing and possible branch-flush penalty.

.text

main:
    li    t0, 1           # first input value
    li    t1, 2           # second input value
    li    s0, 3           # loop counter: execute the loop 3 times

loop:
    # Five independent additions.
    # These instructions do not depend on one another, so a pipeline can
    # overlap their execution.
    add    t2, t0, t1      # t2 = 1 + 2 = 3
    add    t3, t0, t1      # t3 = 1 + 2 = 3
    add    t4, t0, t1      # t4 = 1 + 2 = 3
    add    t5, t0, t1      # t5 = 1 + 2 = 3
    add    t6, t0, t1      # t6 = 1 + 2 = 3

    addi   s0, s0, -1      # decrement loop counter
    bnez   s0, loop        # if s0 != 0, jump back to loop

    # Deliberate NOPs: no useful work is performed here.
    # They make the instructions after the branch easy to see in Ripes.
    nop
    nop
    nop
    nop

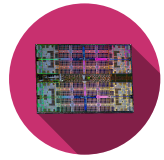
    ecall                 # terminate program
```



1. Iterate the code step by step. At each occurrence of the jump instruction **bnez s0, loop**, observe the behavior of the pipeline. What happens?
2. When arriving for the last time at the jump instruction, observe the behavior of the pipeline. What happens differently?
3. Explain why the pipeline is not “5 times faster” in practice.



This chapter introduces advanced concepts used today to optimize processor architectures. The goal is to introduce the idea of pipelining and its practical limits, without analyzing all the micro-architectural details.



A Full-size LW datapath

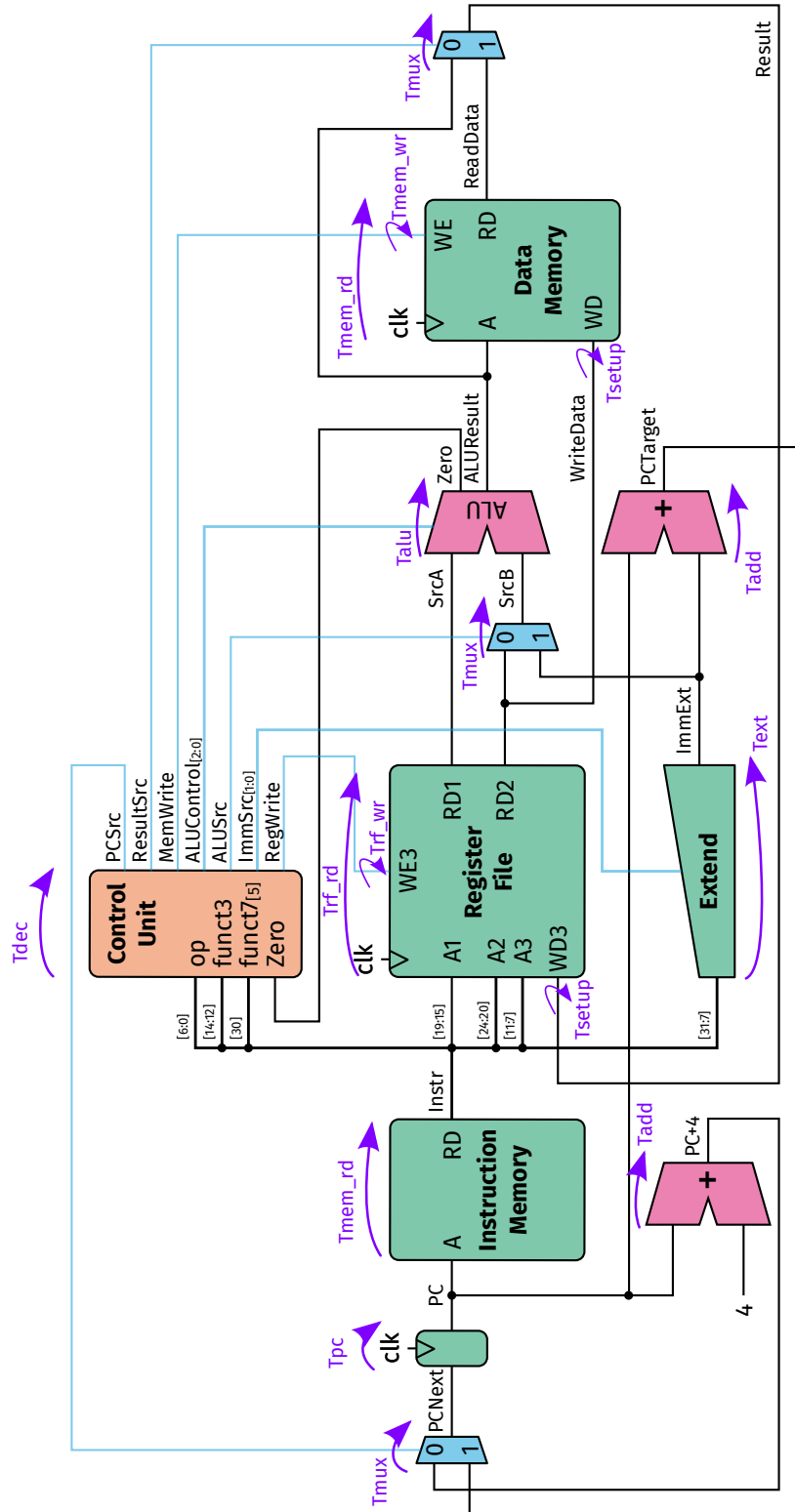


Figure 6 - Full-size reference datapath for LW timing